Sesión 6

43

Lo nuevo filo recibe b=4
Porque AUTO_INCREMENT en MySQL no reutiliza ids de borrados.
los tres primeros fantasy tomaron 1, 2 y 3; el delete de 3 no bajo el contador, el siguiente INSERT tomo 4 (aunque la tabla solo tiene ids 1, 2, 3)

Sesión 6 (44)

Muestra 1 solo año
Porque el left join conserva a 5 NULL, pero al tener conjes genero una fila con NULL en los calculo del historial - conjes cuenta) cuanto final no valores no nulos, osea cuanto esta fila NULL -> 1 (si fuera COUNT(h.id - conje), daria 0.)

Sesion 6 (45)

Devuelve 3 filas. Por que DISTINCT aplica a la combinacion (genero, precio):
- accion + 0.00 - 1 (Fortnite y free fire colapsan en uno)
- Deportes + 149.90 - 1 (FIFA 26)
- Deportes + 0.00 -> (Rocket league)
Total tiene 3 filas. No es 4 los dos de accion a 0.00 son identicos en ambos columnas.

Sesion 6 (46)

Fallo con error de integridad referencial (FK)
Por que la fk de historial_conjs hacia estudiantes se definio sin ON DELETE osea que la accion por defecto es RESTRICT. MySQL impide borrar un estudiante que tiene conjes referenciandolo. hay que borrarlo primero los conjes (o definir ON DELETE CASCADE / SET NULL).

Sesion 8 (47)

A -> /tienda/juegos/buscar, B -> /buscar.
Por que una accion relativa (buscar) se resuelve contra la url actual (/tienda/juegos/buscar.HTML) - /tienda/juegos/buscar una accion que empieza por / es absoluta desde la raiz del dominio (no del contexto.) -> /buscar perdiendo el contexto /tienda

Sesión 6 (48)

Devuelve 3.5 y 3.
Porque en MySQL, / es división decimal → 7/2 = 3.5, DIV es división entera → 7 DIV 2 = 3

Sesión 7 (49)

Imprime 1 (el valor de id_Estudiantes)
Porque: rs.getString(1) lee la primera columna del SELECT (id_Estudiante) como texto. El 1 es el índice de columna, no el valor. Para leer el nombre habría que usar rs.getString(2) o rs.getString("nombre")

Sesión 6 (50)

Falla con error Data too long for column 'n' (modo estricto)
Porque varchar(5) solo admite 5 caracteres y 'carlos Mendoza' tiene 14. Con STRICT_TRANS_TABLES (predeterminado en MySQL 8 y MariaDB), MySQL rechaza el INSERT entero de forma que si modo estricto se guardara 'carlo' con un warning.

Sesión 6 (51)

bono = 1200.
Porque en UPDATE las asignaciones se evalúan de izquierda a derecha usando los valores ya actualizados de columnas anteriores. Primero puntos = 500 + 100 = 600; luego = puntos * 2 usa el nuevo puntos (600) → bono = 1200.

Sesión 8 (52)

Funciona sin error: la cookie se añade a la respuesta
Porque new Cookie(...) + response.addCookie(c) es la api estándar de Jakarta.servlet.http.Cookie y sigue vigente en Tomcat 11, no hay problema con el espacio en el valor "Una torre" ni con el punto y coma. La cookie llega al navegador. (Si el valor tuviera caracteres no válidos, Tomcat lanzaría IllegalArgumentException, pero aquí es válido.)

Sesión 5 (53)

Init se imprime 1 vez; doGet se imprime tres veces.
Por que: init() se ejecuta solo una vez cuando el contenedor crea la instancia del servlet (la primera petición o al arrancar, según load-on-startup). doGet se ejecuta en cada petición. 3 peticiones + init + 3 get

Sesión 6 (54)

Al borrar a carlos también se borra sus conjes en historial_conjes (en cascada), y el delete de carlos se completa
Por que ON DELETE CASCADE propaga el borrado a los (hijos hijos) a diferencia de la FK sin ON DELETE (que bloqueaba con RESTRICT) aquí MySQL eliminó primero los conjes de carlos y luego a carlos y como resultado ni carlos ni sus conjes desaparecen

Sesión 8 (55)

${mensaje} sale vacío (no muestra "bienvenido").

Por que sendRedirect genera nueva petición HTTP al navegador hacia el menu.jsp los atributos de request no sobreviven a un redirect (son de la petición original). En menu.jsp, ~~atributos~~ ${mensaje} es null => El no muestra como cadena vacío

Sesión 8 (56)

LLegan solo juegos (el hidden). Codigo y grado No llegan
Por que
- Disabled → El campo no se envían solelo
- readonly → El campo sí se envía normal existe
- Grado=5 sí llega
- hidden → se envía siempre → Juego=3 sí llega

Sesión 7 (57)

Imprime true

Por que: rs.getINT(1) sobre NULL devuelve 0 (valor por defecto de int en JDBC). Luego rs.wasNull() devuelve true, indicando que el último valor leído era NULL, por eso: 0 true.

MARK BOOK

Sesión 7 (59)

Fallo rs.next() lanza SQLException (OperationNotAllowedAfterResultSetClosed/ConnectionClosed).
Porque: El try-with-resources cierra conexión y preparedStatement al salir del método, y con ellos el ResultSet que devolvió executeQuery().

Sesión 8 (60)

El servidor lanza NullPointerException (error 500)
Porque: si la casilla no se marca, el navegador no envía el parámetro acepto. request.getParameter("acepto") devuelve Null, y Null.equals("on") lanza NullPointerException. Hay que invertir la comparación.

Sesión 6 (61)

Devuelve Fornite y Free Fire (2 filas).
Porque: la colación utf8mb4_unicode_ci es case_insensitive (y accent_insensitive). 'acción' coincide con 'accion' (mayúsculo y Tilde no importan). Por eso devuelve los dos juegos

Sesión 6 (62)

Devuelve Fifa 26 y gran turismo 7 (2 filas)
Porque: Order By Precio DESC ordena: Zeldo 259 Mario Kart 8 219, Fifa 26 199.90, gran turismo 7 189
LIMIT 1,2 significa: saltar 1 fila (Zeldo) y tomar 2 - Fifa 26 y gran turismo 7.

Sesión 8 (63)

NO Compila
Por que hay dos problemas
- Faltan los import (java.sql.Connection, java.sql.DriverManager, java.sql.SQLException), sin ellos conexión y DriverManager no se reconocen -> error de compilación
- DriverManager.getConnection(...) lanza SQLException (checked): El método debe declararla en throws o capturarla con try/catch. doGet solo declara ServletException IOException -> error de compilación

Sesión 7 (64)

rs.next() lanza SQLException (Operation not allowed after ResultSet closed / statement closed).
Por que un statement solo puede tener un ResultSet abierto al a vez. Al ejecutar rs2 = st.executeQuery(...) JDBC cierra automaticamente el resultset anterior (rs1). cuando llamas rs1.next(), el cursor ya esta cerrado - excepción. Solución: usar dos statement distintos, o procesar rs1 antes de ejecutar rs2.

Sesión 8 (65)

con forward (opción 1) la barra sigue mostrando la url del servlet original. con sendRedirect (opción 2) la barra cambia al url de menu.jsps
Por que forward es interno al servidor: El navegador no se entera, o URL no cambia

Sesión 8 (66)

Lanza IllegalStateException (cannot call sendRedirect() after The response has been committed).
Por que out.flush() envia la respuesta al cliente y la marca como committed (ya no se puede modificar la cabecera).

Sesión 8 (67)

Devuelve Null
Por que request.getSession(false) no crea una sesión nueva; solo devuelve la existente. Si el visitante nunca inicio sesión (no hay JSESSIONID valido), devuelve null. (con getSession() o getSession(true) si se crearia una)

sesión 6 (68)

Orden: Luis Rojas (200), Carlos mendoza (500). Ana falta (Null). Por que ORDER BY puntos Asc ordena de menor a mayor
corrección orden real de MySQL Asc Null primero (null), Luis Rojas (200), carlos mendoza (500).

Sesión 8 (69)
G = "acción" (solo el primer valor).
Por que: request.getParameter("genero") devuelve el primer valor aunque hay varios con el mismo nombre. Para obtener todos hay que usar request.getParameterValues("genero") que devuelve un String[] con {"accion", "deportes"}.

sesion 8 (70)
Fallo el despliegue no compila: la clase no extiende HttpServlet. Por eso el codigo esta mal escrito: la clase LoginServlet no extiende HttpServlet (el extends esta fuera de la clase en una linea suelta. un servlet debe ser public class LoginServlet.

Sesión 6 (71)

Devuelve 0 filas (vacio)
Porque: NOT IN con una subconsulta que tiene null se comporta de forma especial: x not in (1, 2, Null) equivale a x <> 1 AND x <> 2 AND x <> Null como x <> Null es Null) desconocido, toda la condicion resulta desconocido → ninguna fila cumple

sesion 7 (72)
Getgeneratedkeys() lanza SQLexception (o devuelve un resulset vacio) Porque por defecto preparestatement (sql) no activa la devolución de claves generadas hay que crear el PreparedStatement con la constante

Sesion 8 (73)

La contraseña viaja en la url (visible en la historial, logs, proxies, Referer). por eso method="get" envia los parametros en la Query String (?usuario=...&clave...). la contraseña queda expuesta en la barra de direcciones, en el historial del navegador en los logs del servidor y en cabeceras Referer. Deberia el get debe ser independiente

Sesion 8 (74)

Tomcat responde con 404 not found.
Por que: El mapeo "/routing" es exacto: solo coincide con la url /routing. La url /Routing/ no coincide con el Patron exacto. Tomcat no encuentra el servlet → 404. para que routing/ funcione habria que mapear "/routing/*" o "" segun el caso

Sesion 6 (75)

Fallo con SQL Exception: Subquery returns more than 1 row.
Por que la sub consulta (select edita puntos FROM recompensas) devuelve 2 filas (150 y 300). El operador = espera solo un valor escalar: al recibir 2 filas, MySQL lanza error. Para comparar con varios valores de usuario → IN (select...)

Sesion 7 (76)

SP completo
Por que dado que, el try with-resources permite usar una variable ya declarada (try (con) {...}) siempre que sea final o efectivamente final. connection implementa autocloseable, así que es valido. El con se cierra automaticamente al salir del bloque

Sesion 7 (77)

Fallo con SQL Exception (error de sintaxis SQL).
Por que en JDBC, los ? de prepared statement solo sirven para valores, no para identificadores (nombres de tabla o columna). select * from ? Genera SQL invalido (select * from 'usuarios'). Para una tabla dinamica hay que validar el nombre con whitelist y concatenarlo

Sesión 7 (78)

Falla de conexión: Unknown database 'gomas_store_DB'
Por que en Linux (y en MySQL), los nombres de base de datos son case-sensitive por defecto (lower_case_table_names=0). La base real es gomas_store_db (minúscula). Como Tore_DB no existe con esos mayúsculos. MySQL no lo encuentra

Sesión 7 (79)

Imprime el nombre del primer estudiante (por ejemplo, carlos mendoza). Por que if (rs.next()) mueve el curso al primero feliz y devuelve true si hay al menos uno, como la tabla tiene 3 estudiantes, entro al if y rs.getString("nombre") lee el nombre de ese primero feliz (el order sin ORDER BY es indefinido, pero típicamente el de inserción: carlos mendoza.

Sesión 6 (80)

Devuelve 350

Por que SUM(costo) ignora los NULL. Suma 500+200=700 y divide entre 2 filas con valor (no entre 3) → 350. Si se quisiera dividir entre todas las filas (3), habría que usar SUM(puntos) / count(*) = 700/3 = 233.33